Running Ad Hoc Commands

Ansible Fundamentals

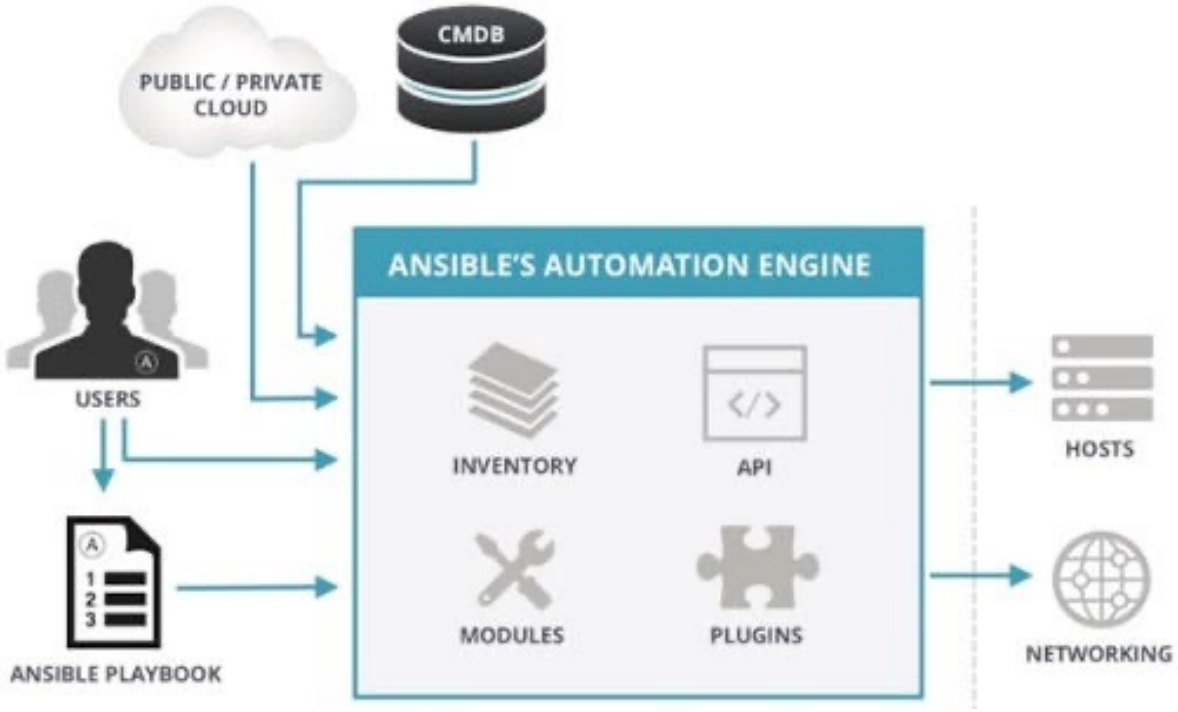
Agenda

- Ansible Architecture
- Ansible Configuration File
- Ansible command syntax
- Run ad-hoc commands
- Authenticating Ansible Connections

Ansible Architecture

Running Ad Hoc Commands

Ansible Architecture



Automation Engine: Inventory

- By default, Ansible represents what machines it manages using a very simple INI file creating a static inventory, using groups of your own choosing
- Other Ansible option, is to use a dynamic inventory to pull your inventory from data sources like AWS, Azure, vSphere, etc.
- Once inventory hosts are listed, variables can be assigned to them in simple text files (in a subdirectory called 'group_vars/' or 'host_vars/' or directly in the inventory file.

Automation Engine: Inventory

- Example of a inventory file

```
---
[webservers]
www1.example.com
www2.example.com

[dbservers]
db0.example.com
db1.example.com
```

Automation Engine: Modules

- Building Blocks of Tasks: Modules are the fundamental units in Ansible used to perform individual tasks. When you define a task in a playbook, you are essentially calling a module with specific parameters.
- Wide Range of Functions: Ansible comes with a vast library of built-in modules that can manage system resources, software packages, files, cloud infrastructure, network devices, and much more.
- Idempotent Operations: Like playbooks, most modules are designed to be idempotent. This means that they will only make changes if the current state of the system doesn't match the desired state.
- Extensibility: While Ansible provides a plethora of built-in modules, users can also write custom modules to cater to specific needs or to manage unique systems.
- Language Agnostic: Although Ansible is written in Python, modules can be written in any language. As long as the language can return JSON, it can be used to write an Ansible module.

Modules: yum package manager

- Used for managing packages with the yum package manager, which is common on Red Hat-based systems.

```yaml
tasks:
  - name: Install nginx
    yum:
      name: nginx
      state: present
```

Modules: file

- Manages file properties, such as permissions, ownership, and links

```yaml
tasks:
  - name: "Change ownership of /etc/foo.conf to user 'john' and group 'john'"
    file:
      path: /etc/foo.conf
      owner: john
      group: john
```

Modules: community.vmware.vmware_guest

- modify an existing VM using the vmware_guest module. In this scenario, we'll adjust the CPU and memory configuration of an existing VM.

```yaml
tasks:
  - name: Adjust VM hardware configuration
    community.vmware.vmware_guest:
      hostname: 'vcenter.example.com'
      username: 'administrator@vsphere.local'
      password: 'your_password'
      validate_certs: no
      cluster: 'ClusterName'
      name: 'existing_vm_name'
      hardware:
        memory_mb: 4096
        num_cpus: 4
        hotadd_cpu: yes
        hotadd_memory: yes
      state: 'present'
```

Automation Engine: Plugins

- Extensions to Core Features: Plugins augment Ansible's core functionality. They are small pieces of code that enhance built-in capabilities, allowing Ansible to interact with various systems, software, and APIs.
- Diverse Categories: There are many types of plugins in Ansible, including inventory, action, callback, connection, filter, lookup, and more, each serving a specific purpose.
- Customization: While Ansible comes with a wide range of built-in plugins, users can also develop custom plugins to meet specific needs or to integrate with unique systems.
- Execution: Unlike modules, which are executed on the target machine, plugins typically run on the control machine, enhancing the functionality of the Ansible core engine.
- Configuration: Plugins can be configured within the ansible.cfg file, allowing users to define specific settings or choose which plugins to use for specific tasks.

Plugins: Examples

- Inventory Plugins
  - Purpose: Dynamically generate inventory from external data sources.
  - Example: The aws_ec2 inventory plugin fetches inventory from AWS EC2.
- Callback Plugins
  - Purpose: Enable users to add additional actions to respond to Ansible events.
  - Example: The profile_tasks callback plugin provides a summary of how much time each task took.
- Lookup Plugins
  - Purpose: Retrieve data or variables from external sources.
  - Example: The password lookup plugin can be used to generate random passwords for use in playbooks.

Automation Engine: Modules vs Plugins

- Ansible Modules
  - Purpose: Modules are the units of work in Ansible. They are the commands or set of similar commands executed on the target machine(s).
  - Execution Location: Modules are executed on the target machine(s). The control machine sends the module to the target, where it's executed.
  - Language: While most built-in modules are written in Python, they can technically be written in any language. The key is that they accept and return JSON.
  - Usage: Used within tasks in playbooks to perform actions, manage state, or gather data from target systems.
- Ansible Plugins
  - Purpose: Plugins augment or extend Ansible's core functionality. They modify or enhance the behavior of Ansible but don't directly perform tasks on target machines.
  - Execution Location: Plugins are executed on the control machine, not on the target systems.
  - Language: Plugins are primarily written in Python.
  - Usage: Enhance the functionality of playbooks, configurations, and runtime operations. They can be used to customize Ansible's behavior, integrate with external systems, or process data.

Automation Engine: Playbooks

- **YAML Scripts**: Playbooks are written in YAML (Yet Another Markup Language), which is both human-readable and machine-parsable, ensuring clarity in defining tasks.
- **Ordered Set of Tasks**: Playbooks define an ordered set of tasks to be executed against host machines, ensuring configurations are applied in the correct sequence.
- **Idempotent Operations**: Tasks in playbooks are designed to be idempotent, meaning they can be executed multiple times without changing the result after the first successful run.
- **Multi-Tier Orchestration**: Playbooks can manage configurations across multi-tier setups, allowing for complex deployments involving databases, application servers, web servers, etc.
- **Variables & Templates**: Playbooks support the use of variables and templates, enabling dynamic configurations based on specific conditions or inputs.

Automation Engine: Playbooks

- **`hosts: webservers`**: This playbook targets hosts grouped under "webservers" in the Ansible inventory.
- **`vars`**: Defines variables (http_port and max_clients) that can be used later in the playbook or in templates.
- **`tasks`**: Lists the tasks to be executed in order. Here, it ensures Apache is installed, running, and a custom configuration file is uploaded.
- **`yum and service modules`**: These are used to manage packages and services on the target hosts, respectively.
- **`template module`**: This is used to push a templated configuration file to the target hosts. The .j2 extension indicates it's a Jinja2 templated file, which can utilize the defined variables for dynamic content.
- **`handlers`**: These are special tasks that run once, at the end of the playbook, if notified by another task. In this case, if the configuration file changes, Apache will be restarted.

```yaml
---
- name: Install and Start Apache Web Server
  hosts: webservers
  vars:
    http_port: 80
    max_clients: 200
  tasks:
    - name: Ensure Apache is installed
      yum:
        name: httpd
        state: present

    - name: Ensure Apache is running and enabled at boot
      service:
        name: httpd
        state: started
        enabled: yes

    - name: Upload custom configuration file
      template:
        src: /path/to/local/template.cfg.j2
        dest: /etc/httpd/conf/custom.cfg
      notify: Restart Apache

  handlers:
    - name: Restart Apache
      service:
        name: httpd
        state: restarted
```

Ansible Configuration File overview

Introduction to Ansible

Configuration File

- **Unified Settings**: The Ansible configuration file, ansible.cfg, provides a consolidated location to define settings that dictate how Ansible operates and interacts with different systems.
- **Hierarchy of Precedence**: Ansible can have multiple configuration files, and they are processed in a specific order of precedence, allowing for both global and project-specific settings.
- **INI Format**: The configuration file uses the INI format, making it easy to read and edit. Sections are defined using square brackets, and key-value pairs within those sections set various configurations.
- **Default Location**: By default, Ansible looks for the configuration file at **`/etc/ansible/ansible.cfg`**, but this can be overridden by user-specific or project-specific configuration files.
- **Flexibility**: The configuration file allows users to modify a wide range of parameters, from specifying the inventory path, adjusting parallel task execution, setting timeout values, to defining custom plugins or modules paths.

Places for Configuration

- **`ANSIBLE_CONFIG`** Environment Variable
  - An environment variable that points to the location of the config file.
- Current Directory
  - **`ansible.cfg`** in the current directory from which ansible or ansible-playbook is run.
- Home Directory
  - **`.ansible.cfg`** in the user's home directory.
- Global Configuration
  - **`/etc/ansible/ansible.cfg`**

Some configurations

- **`inventory`**: Specifies the location of the inventory file, which contains a list of the nodes that Ansible manages.
  - Example: **`inventory = /etc/ansible/hosts`**
- **`remote_user`**: Default username used to connect to target machines.
  - Example: **`remote_user = admin`**
- **`host_key_checking`**: Determines if Ansible checks the remote host's SSH key. Turning this off is useful for managing a large number of hosts without initial manual intervention.
  - Example: **`host_key_checking = False`**
- **`forks`**: Defines the number of parallel processes to use when communicating with remote hosts. It essentially controls parallelism.
  - Example: **`forks = 10`**
- **`log_path`**: Specifies the location where Ansible should write its log file. This is useful for troubleshooting and auditing purposes.
  - Example: **`log_path = /var/log/ansible.log`**

Ansible command syntax

Introduction to Ansible

CLI Commands

- `ansible`
- `ansible-config`
- `ansible-console`
- `ansible-doc`
- `ansible-inventory`
- `ansible-playbook`
- `ansible-pull`
- `ansible-vault`

CLI Commands: **`ansible`**

- This command allows you to execute ad-hoc commands on target hosts
- Example: This pings all hosts in your inventory to check if they are reachable.

```
ansible all -m ping
```

CLI Commands: `ansible-config`

- Allows you to view, list, and manage Ansible configuration
- Example: This displays the current Ansible configuration

```
ansible-config view
```

CLI Commands: `ansible-console`

- Provides an interactive REPL (Read-Eval-Print Loop) interface for executing ad-hoc commands
- Example: This starts the interactive Ansible console

```
ansible-console
```

CLI Commands: **`ansible-doc`**

- Provides documentation on Ansible modules
- Example: This shows documentation for the **`yum`** module.

```
ansible-doc yum
```

CLI Commands: `ansible-inventory`

- Allows you to view and manage the Ansible inventory
- Example: This lists all hosts in the current inventory

```
ansible-inventory --list
```

CLI Commands: **`ansible-playbook`**

- Executes Ansible playbooks, which are scripts that define a set of tasks to be run on target hosts
- Example: This runs the **`deploy_app.yml`** playbook.

```
ansible-playbook deploy_app.yml
```

CLI Commands: **`ansible-pull`**

- A mode that inverts the default push architecture of Ansible into a pull architecture, which can be useful for scalable or decentralized setups
- Example: This pulls a playbook from a Git repository and executes it locally

```
ansible-pull -U https://github.com/user/repo.git
```

CLI Commands: **`ansible-pull`**

- Provides encryption and decryption capabilities for sensitive data in playbooks or variables files
- Example: This encrypts the **`secrets.yml`** file.

```
ansible-vault encrypt secrets.yml
```

Run ad-hoc commands

Introduction to Ansible

Ad-Hoc Commands

- An ad hoc command uses the **`/usr/bin/ansible`** command-line tool to automate a single task on one or more managed nodes.
- Ad hoc commands are quick and easy, but they are not reusable.
- Ad hoc commands demonstrate the simplicity and power of Ansible
- The concepts you learn here will port over directly to the playbook language. Before reading and executing these examples, please read

Why Using Ad-hoc Commands?

- Ad hoc commands are great for tasks you repeat rarely
- For example, if you want to power off all the machines in your lab for Christmas vacation, you could execute a quick one-liner in Ansible without writing a playbook
- An ad hoc command looks like this:

```
$ ansible [pattern] -m [module] -a "[module options]"
```

- The **`-a`** option accepts options either through the **`key=value`** syntax or a JSON string starting with { and ending with } for more complex option structure

How to identify machines

- Ad hoc commands uses inventories to define the machines to be reached
- Inventory may be referenced in different ways following this precedence list
  - Command Line **`-i`** Option
  - **`ANSIBLE_INVENTORY`** Environment Variable
  - **`ansible.cfg`** Configuration File:
  - Default Inventory Path: **`/etc/ansible/hosts`**
- Inventory file defines groups of hosts and identify them with a name
- When executing the command you use patterns to define which machines you will run the command

Common Patterns

Description	Pattern(s)	Targets
All hosts	all (or *)	
One host	host1	
Multiple hosts	host1:host2 (or host1,host2)	
One group	webservers	
Multiple groups	webservers:dbservers	All hosts in webservers plus all hosts in dbservers
Excluding groups	webservers:!atlanta	All hosts in webservers except those in atlanta
Intersection of groups	webservers:&staging	Any hosts in webservers that are also in staging
Control node	localhost	Run command on control node only

Simple ad-hoc commands

- Reboot all servers on **`web`** group on inventory

```
$ ansible web -a "/sbin/reboot"
```

- Ansible runs 5 execution simultaneous. If you want to execute 10, you use flag **`-f`**

```
$ ansible web -a "/sbin/reboot" -f 10
```

Simple ad-hoc commands

- If you want to execute the command with another username, you may use **−u** flag

```
$ ansible atlanta -a "/sbin/reboot" -f 10 -u username
```

- And sometimes you need to elevate your user

```
$ ansible atlanta -a "/sbin/reboot" -f 10 -u username --become [--ask-become-pass]
```

Run ad-hoc commands using modules

- On previous samples, you're using the **`ansible.builtin.command`**
- This is the default module and you don't need to specify on every command
- The flag **`-a`** represent the argument for module command, that on this case represent the command itself
- To specify a different module you may use **`-m`** flag
- Module name can be composed by several blocks using a dot to separate them
- When you do use this full name, the module will be related with **`ansible.builtin`**

Run ad-hoc commands using modules

- Ping all machines

```
$ ansible all -m ping
```

- Copy **/etc/hosts** to **/tmp/hosts** on atlanta group within the inventory

```
ansible atlanta -m ansible.builtin.copy -a "src=/etc/hosts dest=/tmp/hosts"
```

Run ad-hoc commands using modules

- You may create folders with several parameters

```
ansible webservers -m file -a "dest=/path mode=600 owner=user group=group"
```

- Restart **`httpd`** service on **`webservers`** group within the inventory

```
$ ansible webservers -m service -a "name=httpd state=restarted"
```

Run ad-hoc commands with explicit inventory

- Using **`-i`** flag to define inventory

```
$ ansible -i inventory/dev.yml webservers -m service -a "name=httpd state=restarted"
```

Run Ad-Hoc Commands

Demo

Authenticating Ansible Connections

Introduction to Ansible

How to authenticate

- SSH Keys
  - Ansible primarily uses SSH keys for authentication
  - It's the most common method where the control machine has a private key and the managed nodes have the corresponding public key in the authorized keys list
- Username & Password
  - While less secure and not recommended for production, Ansible can use SSH with a username and password for authentication
  - This method can be useful in scenarios where key-based authentication isn't feasible
  - You need to have **`sshpass`** installed on control node

How to authenticate

- Become (Privilege Escalation)
  - For tasks that require elevated privileges, Ansible uses the become method.
  - This can leverage tools like sudo, su, pbrun, and others to gain higher-level permissions.
- SSH Configuration & ssh-agent
  - Ansible can leverage existing SSH configurations and keys loaded into the ssh-agent for authentication.
  - This allows users to use jump hosts, non-default ports, and other SSH settings.
- Vault for Encrypted Credentials
  - Ansible Vault can encrypt sensitive data, including authentication credentials.
  - This ensures that secrets are stored securely but can be decrypted by Ansible during playbook runs.

SSH Key

- Ansible leverages the native SSH mechanism for authentication, which means it uses the SSH keys already defined on the control node (the machine running Ansible) to authenticate to remote servers
- Default Private Key
  - By default, Ansible uses the private key located at **`~/.ssh/id_rsa`**
- Specifying a Different Key
  - If you want to use a different private key, you can specify it using the **`--private-key`** option or set it in the Ansible configuration file (**`ansible.cfg`**) using the **`private_key_file`** setting.
- ssh-agent
  - If you're using **`ssh-agent`** on the control node, Ansible can leverage it.
  - When you add your private keys to the agent using **`ssh-add`**, Ansible will use the identities loaded into the agent for authentication.
- Ansible Vault
  - If you need to store SSH private keys securely, you can encrypt them using Ansible Vault.
  - When running playbooks, you can decrypt the key on-the-fly, ensuring that sensitive keys are not exposed in plaintext.